Asistente Inteligente para Estudiantes Duoc UC: agente con herramientas, memoria y planificación

Integrantes: Estephany Cárdenas · Yannella Castilla | Duoc UC, noviembre de 2026

Repositorio: https://github.com/Yannnella/asistente_estudiantes_duoc (README con instrucciones de ejecución, diagramas y evidencias)

En la Evaluación Parcial N°1 construimos un sistema multiagente con RAG que responde consultas de estudiantes de Duoc UC solo con documentos institucionales, cita la fuente y deriva a una persona cuando no hay evidencia. Respondía bien preguntas aisladas, pero muchas necesidades reales son procesos de varios pasos que duran días: para iniciar la práctica profesional hay que verificar el avance de malla, conocer los documentos, revisar si la inscripción está abierta, preparar la solicitud y retomar el trámite más adelante. En esta segunda parte el sistema se convierte en un **agente** que sigue el ciclo percepción → razonamiento → decisión → acción: usa herramientas de consulta, razonamiento y escritura, recuerda el contexto dentro de una sesión y entre sesiones, y ajusta su plan cuando cambian las condiciones.

1. Diseño e implementación del agente (IE1, IE2)

El agente se implementó con **LangChain** (Chase, 2022), el framework de agentes individuales visto en clase, sobre la misma base del EP1. Las herramientas son `StructuredTool` con argumentos tipados en Pydantic; LangChain los convierte en JSON Schema y el LLM las elige por **function calling** nativo (OpenAI, s. f.), que evita el parseo frágil de texto del ReAct clásico. El LLM es GPT-4o-mini en GitHub Models, con temperatura 0.

Orquestación jerárquica. Un **Agente Manager** recibe la solicitud, decide la estrategia, delega cada paso en un agente ejecutor y valida su resultado; no tiene herramientas propias. Los tres ejecutores (Consulta, Razonamiento y Escritura) reciben solo las herramientas que necesitan y no se comunican entre sí. El Manager elige entre tres estrategias: **reactiva**, con el patrón ReAct (Yao et al., 2023) implementado con `AgentExecutor` y `create_tool_calling_agent` (máximo cuatro iteraciones y solo herramientas sin efectos secundarios), para consultas puntuales; **planificada** (Plan-and-Execute; Wang et al., 2023) para trámites de varios pasos, en la que el plan completo se genera antes de actuar con salida estructurada del LLM o, sin LLM, con un planificador por reglas; y **directa** para acciones críticas, como derivar un caso excepcional o borrar datos, que nunca se delegan al LLM. El RAG multiagente del EP1 no se reescribió: quedó encapsulado en `buscar_normativa`, con sus guardrails, el umbral de relevancia y las citas armadas por código.

Por qué LangChain y no CrewAI. El RAG del EP1 ya usa LangChain, y un Crew jerárquico hace varias llamadas al LLM por tarea, mientras la cuota diaria de GitHub Models es baja. El patrón Manager–especialistas se implementó igual, con validación determinista. Los cálculos son código: el LLM decide cuándo calcular y con qué datos, pero no hace la aritmética, siguiendo la idea de delegar en herramientas lo que el modelo hace mal (Schick et al., 2023).

Tabla 1. Agentes ejecutores y herramientas (src/herramientas.py)

Agente	Herramienta	Función
Consulta	buscar_normativa	RAG del EP1 (coordinador + 4 especialistas, ChromaDB) con citas
Consulta	consultar_calendario	Estado de un período (abierto, próximo o cerrado) en la fecha actual
Consulta	consultar_mis_tramites	Trámites, etapas y tickets guardados en la memoria de largo plazo
Razonamiento	verificar_requisitos_practica	Compara el % de malla aprobado con el mínimo de la carrera
Razonamiento	calcular_asistencia	% de asistencia, mínimo exigido (60 % o 70 %) y faltas restantes
Razonamiento	calcular_nota_necesaria	Nota mínima en lo que resta para aprobar con 4,0
Escritura	redactar_solicitud	Borrador en Markdown, sin datos personales
Escritura	registrar_derivacion	Ticket para que una persona atienda el caso
Escritura	guardar_tramite	Etapa y próximo paso de un trámite, para retomarlo en otra sesión
Escritura	olvidar_mis_datos	Borra toda la memoria del estudiante (derecho de supresión)

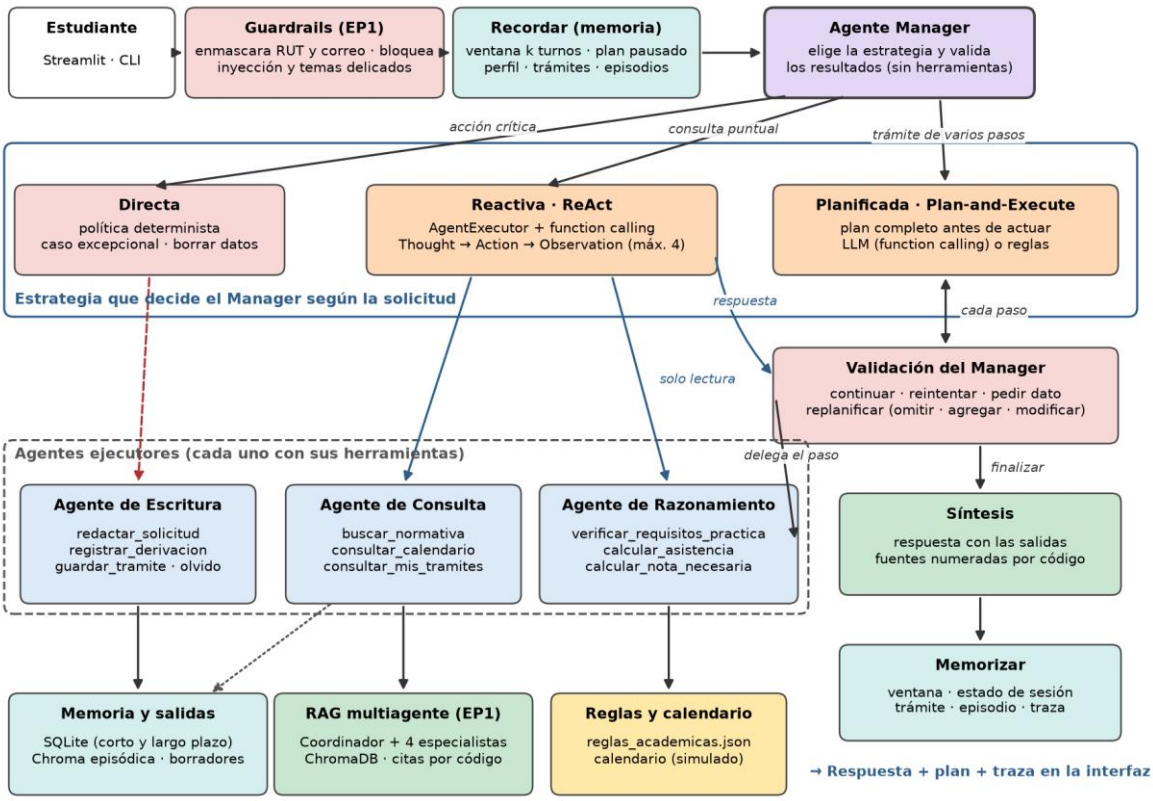


Figura 1. Orquestación jerárquica: el Manager elige la estrategia, delega en los ejecutores y valida cada paso.

2. Memoria y recuperación de contexto (IE3, IE4)

- **Corto plazo (por sesión).** `ConversationBufferWindowMemory` con *k* turnos, inyectada en los prompts del planificador y del agente ReAct mediante `MessagesPlaceholder`; así el agente entiende «¿y cuántas horas dura?». Su `chat_memory` es un `BaseChatMessageHistory` propio sobre SQLite, por lo que la conversación sobrevive a un reinicio. El estado de la sesión guarda el tema activo y el **plan pausado** cuando falta un dato.
- **Largo plazo (por estudiante).** Perfil declarado (carrera, sede, jornada), trámites con su etapa y próximo paso, y tickets, en SQLite. Además, una **memoria episódica** en una colección Chroma propia, recuperada por similitud y filtrada por estudiante (el concepto de `VectorStoreRetrieverMemory`). Al iniciar otra sesión, el agente recuerda los trámites en curso.
- **Elección y privacidad.** Se eligió una ventana (*context pruning*) en lugar de `ConversationSummaryMemory` porque resumir agrega una llamada al LLM por turno; lo que debe durar más se guarda estructurado. Solo se guarda texto enmascarado, las cifras personales se eliminan de la memoria de largo plazo y «Borra mis datos» elimina todo (Ley 19.628 y Ley 21.719).

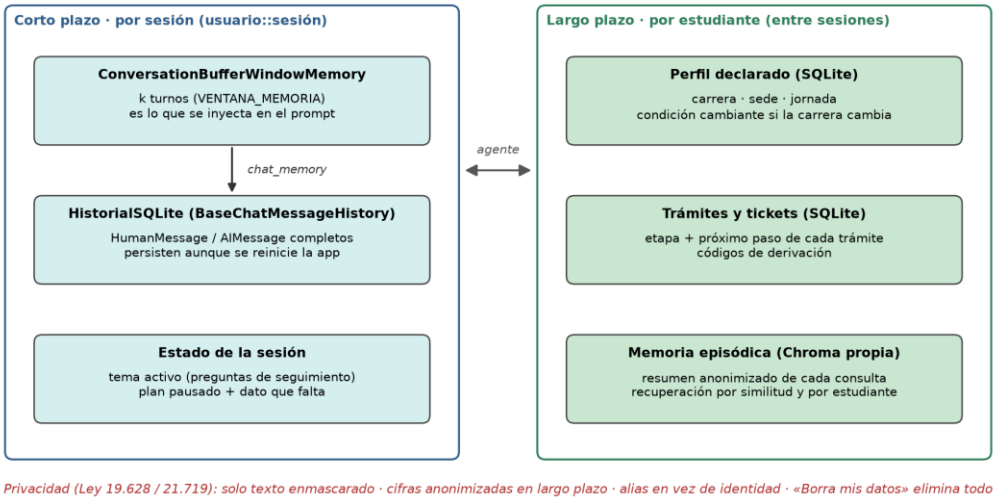


Figura 2. Memoria de corto y largo plazo del agente.

3. Planificación y toma de decisiones (IE5)

Después de cada paso, el Manager valida el resultado y decide continuar, repetir el paso, pausar para pedir un dato o **replanificar**: omitir, agregar o modificar pasos. La política es determinista para que cada decisión sea explicable y se pueda probar. Para controlar costo y latencia, cada turno se limita a 8 herramientas, 3 replanificaciones y un reintento por paso. Así se superan las limitaciones de la planificación básica vistas en clase: el plan no es estático, el ejecutor valida los argumentos y cada paso conoce el resultado de los anteriores.

Tabla 2. Condiciones cambiantes y decisión del agente

Condición observada	Decisión	Evidencia
Falta un dato (carrera, % de malla)	Pausa el plan, pregunta y lo reanuda en el mismo paso	E01
No cumple el avance de malla	Omite calendario y solicitud; etapa «cumplir requisitos»	E02, E11
Inscripción cerrada o aún no abierta	No redacta la solicitud; etapa «esperar apertura»	E03, E14
La carrera no inscribe práctica individual	Omite la inscripción; etapa «coordinar campo clínico»	E05
Los documentos no responden	No responde sin fuente; agrega registrar_derivacion	E08
Asistencia o nota en riesgo	Agrega un paso: qué pasa si reprueba	E06
Falla una herramienta	Repite una vez; si persiste, cancela y deriva con ticket	prueba automática
El LLM propone pasos inválidos o no pedidos	El Manager los descarta antes de ejecutar	prueba automática
La respuesta ReAct no tiene evidencia	El Manager la reemplaza por una derivación	prueba automática

Ejemplo de flujo (escenario E01). El estudiante escribe «Quiero empezar mi práctica, ¿cumplo los requisitos?». El Manager elige la estrategia planificada y arma el plan verificar requisitos → buscar normativa → consultar calendario → guardar trámite. La primera herramienta indica que falta la carrera: el plan se pausa en la memoria de corto plazo y el agente pregunta. Con «Contabilidad General» guarda la carrera en el perfil y reanuda en el mismo paso; como falta el avance de malla, vuelve a preguntar. Con «llevo 72» verifica que cumple (se exige 70 %), obtiene los documentos con su fuente, confirma que la inscripción está abierta y guarda la etapa «reunir documentos». En otra sesión, «¿En qué quedé con mi práctica?» recupera esa etapa desde la memoria de largo plazo. Con la misma solicitud en una fecha en que la inscripción está cerrada, el agente no redacta la solicitud y cambia la etapa a «esperar apertura».

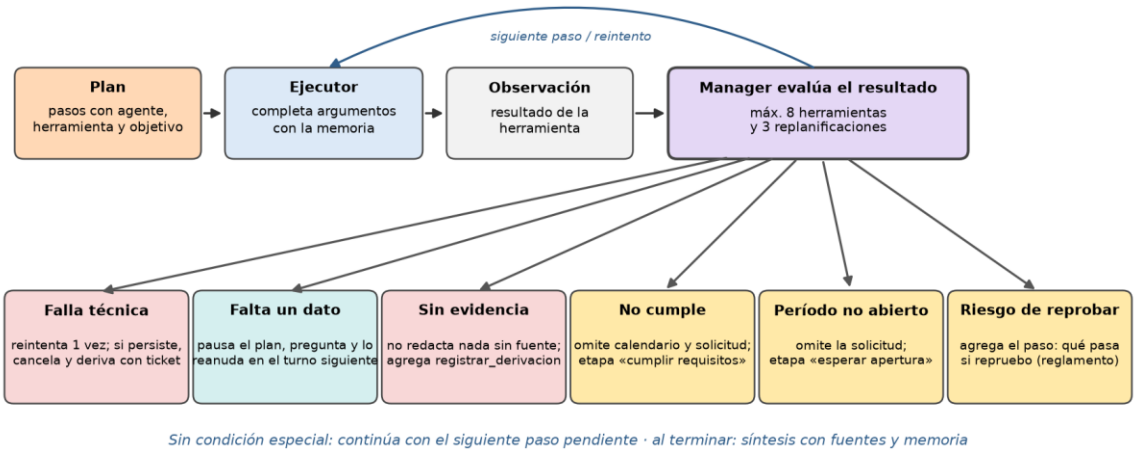


Figura 3. Ciclo ejecutar → observar → validar y decisiones del Manager.

4. Pruebas, resultados y documentación técnica (IE6)

Se implementaron **94 pruebas automáticas** sin red (herramientas, memoria, planificador, agente y las del EP1). Incluyen las rutas con LLM usando modelos de chat simulados: planificación por function calling, `AgentExecutor` llamando herramientas, corrección del Manager cuando la respuesta ReAct no tiene evidencia y cambio a reglas cuando el LLM no responde. Además, **15 escenarios multi-turno** (21 turnos) cubren cada condición cambiante; `scripts/evaluar_agente.py` verifica la estrategia, las herramientas, la replanificación, la pausa, la derivación, la etapa guardada y el contenido de la respuesta.

Tabla 3. Resultados (06-11-2026)

Evaluación	Resultado
Pruebas automáticas (pytest)	94 / 94 aprobadas
Escenarios con RAG simulado	15 / 15 escenarios · 21 / 21 turnos correctos
Escenarios con el índice real, sin token	15 / 15 escenarios · 21 / 21 turnos · 8 replanificaciones · 131 ms por turno · 0 llamadas al LLM
Escenarios con GPT-4o-mini	[completar tras ejecutar scripts/evaluar_agente.py con el token]

El **README** documenta la arquitectura, el rol, el objetivo y las herramientas de cada agente, el modelo y los prompts (few-shot con salida estructurada para el Manager; rol y reglas para el agente ReAct), los procesos de orquestación, la memoria, la política de decisión, la guía de instalación y la tabla de decisiones de diseño con sus alternativas descartadas. Incluye estos diagramas, un diagrama Mermaid del flujo y uno de secuencia del trámite de práctica. La evidencia (salida de pytest, transcripciones de los escenarios con la traza completa y capturas de la interfaz) está en `evidencia/`.

Limitaciones y trabajo futuro. El corpus, el calendario y las reglas son simulados; el alias no es una autenticación y la intención se detecta con reglas. Como mejora, exponer las herramientas por MCP, permitir que otros agentes institucionales deleguen tareas por A2A, empaquetar el sistema en Docker y registrar trazas en LangSmith.

5. Declaración de uso de IA, conclusiones y referencias

Uso de IA. Se utilizó Claude (Anthropic) como apoyo para programar el agente, las pruebas, los diagramas y un borrador de este informe; GitHub Models (GPT-4o-mini) es el LLM del sistema. El equipo revisó y validó el código y los resultados. Uso declarado según https://bibliotecas.duoc.cl/ia.

Conclusiones del equipo y reflexiones individuales

Estephany Cárdenas: [reflexión individual, redactada por la integrante]

Yannella Castilla: [reflexión individual, redactada por la integrante]

Referencias

Chase, H. (2022). *LangChain* [Software]. GitHub. https://github.com/langchain-ai/langchain

Chroma. (s. f.). *Chroma documentation*. https://docs.trychroma.com

Duoc UC. (2025). *2.1.1 Arquitectura y frameworks de agentes; 2.2.1 Sistemas de memoria e integración de herramientas; 2.3.1 Estrategias de planificación y orquestación* [Material de clase]. ISY0101 Ingeniería de Soluciones con IA.

GitHub. (s. f.). *GitHub Models*. https://docs.github.com/en/github-models

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

Ley 19.628 de 1999. Sobre protección de la vida privada. 28 de agosto de 1999. Diario Oficial de la República de Chile. https://www.bcn.cl/leychile/navegar?idNorma=141599

Ley 21.719 de 2024. Regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales. 13 de diciembre de 2024. Diario Oficial de la República de Chile.

OpenAI. (s. f.). *Function calling* [Documentación]. https://platform.openai.com/docs/guides/function-calling

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems, 36*.

Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., & Lim, E.-P. (2023). Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2609–2634.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*.